

\$ W2-1 · 開發工具入門

GITHUB, GIT & DEV TOOLS

開發工具入門 – 版本控制、CI/CD 與基礎程式語言

/> 課程大綱

01

GITHUB

平台介紹、Actions 與 CI/CD

02

GIT

版本控制核心操作

03

GOLANG

語言基礎與模組化架構

04

BASH

Shell Script 入門

\$ PART 01



平台介紹 · GitHub Actions & Runner · CI/CD

➤ GITHUB 是什麼？

建立在 Git 之上的雲端程式碼託管平台

提供版本控制、協作與 Code Review 功能

透過 Pull Request 管理程式碼變更與討論

Issue 追蹤問題，Star / Fork 促進社群協作

全球最大的開源軟體協作社群

➤ GITHUB ACTIONS & RUNNER

GitHub 內建的自動化工作流程引擎

以 YAML 定義 Workflow 步驟

常見 Trigger：push、pull_request

Runner – 實際執行 Workflow 的機器

分為 GitHub-hosted 與 Self-hosted

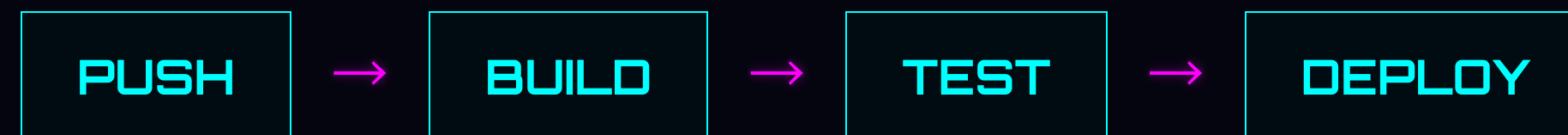
```
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: go build ./...
```

> CI/CD

CI (持續整合) – 自動建置並測試每次提交

CD (持續部署 / 交付) – 自動發布到正式環境

及早發現錯誤、加速開發流程、降低人為疏失



\$ PART 02



版本控制是什麼 · 九個基本操作指令

➤ GIT 是什麼

分散式版本控制系統 (Distributed VCS)

2005 年由 Linus Torvalds 開發，用於維護 Linux Kernel

記錄每次修改的快照，而非差異

支援離線工作、分支管理與多人協作

/> GIT 基本操作 — 本地變更

WORKING DIR

git add



STAGING

git commit



REPOSITORY

`git status` — 查看目前狀態

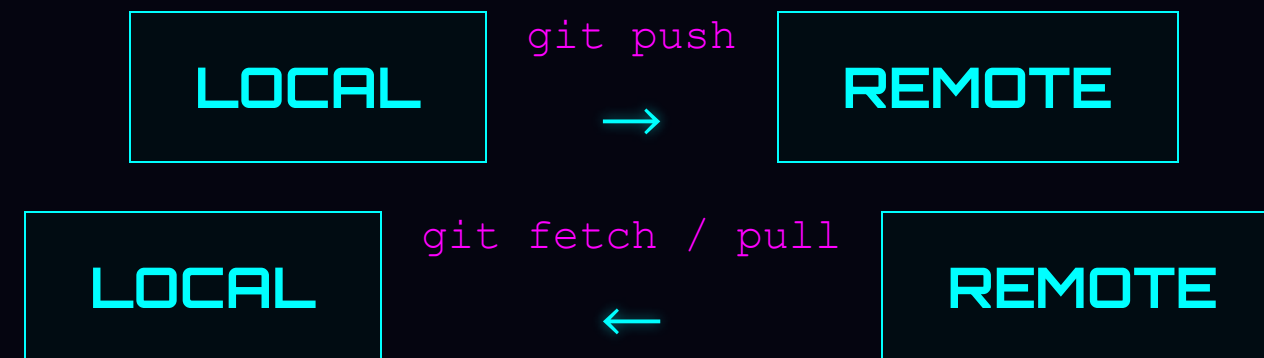
`git add` — 加入暫存區

`git commit` — 建立版本快照

`git log` — 查看提交紀錄

```
$ git status
$ git add .
$ git commit -m "feat: add login
page"
$ git log --oneline
```

➤ GIT 基本操作 — 遠端同步



`git fetch` – 下載遠端資料，不合併

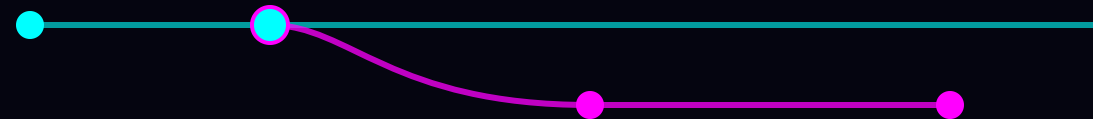
`git pull` – fetch + merge，直接更新

`git push` – 推送本地提交到遠端

```
$ git fetch origin  
$ git pull origin main  
$ git push origin main
```

➤ GIT 基本操作 — 建立分支

Branch



`git branch` – 查看目前所有分支

`git branch name` – 建立新分支

`git checkout -b name` – 建立並切換分支

分支讓多人可同時開發不同功能，互不影響

```
$ git branch
$ git branch feature/login
$ git checkout -b feature/login
$ git switch -c feature/login
```

➤ GIT 基本操作 — 分支整合

Merge



`git merge` — 合併分支，保留完整歷史

`git rebase` — 重新接上提交，歷史更線性

merge 保留脈絡；rebase 更線性

Rebase



```
$ git checkout feature
```

```
$ git merge main
```

```
$ git checkout feature
```

```
$ git rebase main
```

\$ PART 03

GOLANG

語言介紹 · Go Modules · func main() · 測試

GOLANG 是什麼

Google 於 2009 年推出的開源程式語言

靜態型別、編譯式語言，效能接近 C/C++

內建 Goroutine 與 Channel，原生支援併發

語法簡潔、編譯快速，廣泛用於雲端後端服務

➤ GOLANG 的 MOD 化架構

Go Modules – 官方套件依賴管理機制

`go.mod` – 宣告模組名稱與相依版本

`go.sum` – 記錄相依套件雜湊值

```
$ go mod init github.com/user/app  
$ go get github.com/gin-gonic/gin  
$ go mod tidy
```

➤ FUNC MAIN()

可執行程式需有 main 套件與函式

`package main` – 宣告主程式進入點

`func main()` – 程式執行的起始位置

不接受參數，也沒有回傳值

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, CNDI!")
}
```

> TEST

Go 內建測試框架，無需額外套件

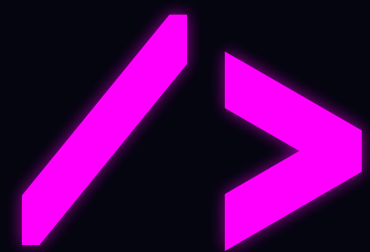
測試檔案以 `_test.go` 結尾

測試函式需以 `Test` 開頭

`go test` – 執行所有測試

```
func TestAdd(t *testing.T) {  
    got := Add(1, 2)  
    if got != 3 {  
        t.Errorf("got %d, want 3",  
got)  
    }  
}
```

\$ PART 04



BASH

Shell Script 入門

➤ BASH 腳本是什麼

Bash – 常見的 Shell 指令解譯器

Shell Script – 可重複執行的指令集

`#!/bin/bash` – 宣告直譯器 (Shebang)

常用於自動化部署與環境設定

```
#!/bin/bash
echo "Deploying project..."
git pull origin main
go build -o app .
./app
```

➤ BASH — 變數、條件與迴圈

`name="value"` — 宣告變數，等號兩側不可留空白

`$name` — 取用變數值

`if [ ]; then fi` — 條件判斷式

`for ... do done` — 迴圈語法

```
#!/bin/bash
name="World"
count=3

if [ -d "./logs" ]; then
    echo "Hello, $name!"
fi

for i in $(seq 1 $count); do
    echo "attempt $i"
done
```

➤ 本章小結

GitHub：雲端協作平台，Actions 讓 CI/CD 自動化

Git：分散式版本控制，本地變更、分支、合併一次搞懂

Golang：靜態型別、原生併發，內建測試框架

Bash：Shell Script 串起自動化部署與環境設定

下一步：把這些工具串起來，實際跑一次完整的開發流程